Universidad Nacional de Colombia - sede Bogotá
Facultad de Ingeniería
Curso: ingeniería de Software II

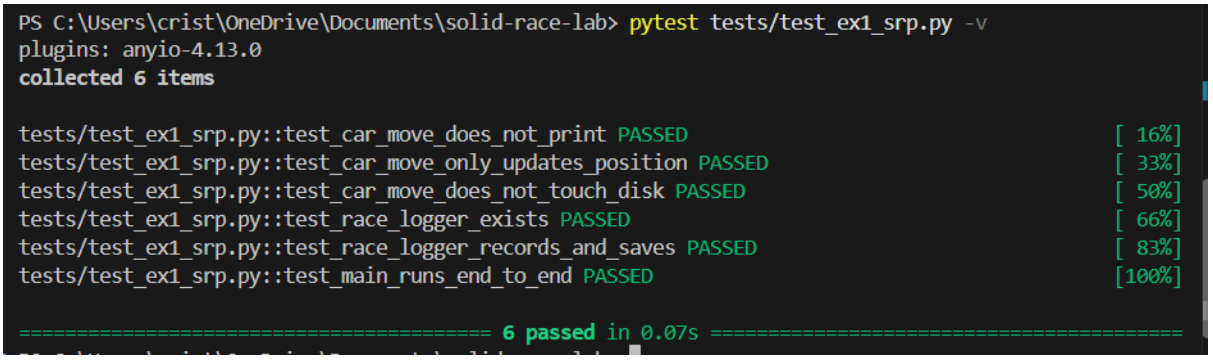
Laboratorio SOLID

Cristian Camilo Ochica Maldonado

Ejercicio 1

La clase original *Car* realizaba tres operaciones distintas que no se relacionaban entre sí, calculaba la posición, imprimía en consola y escribía los resultados en un archivo de texto en el disco. Por lo que para evitar conflictos con el principio de responsabilidad única o SRP se eliminaron las operaciones de consola para que *Car* solo actualice su posición. Además se creó la clase *RaceLogger* para recolectar las posiciones de los vehículos en cada turno, almacenando y guardandola en el disco.

En la función *main* finalmente se maneja la animación y le avisa al logger cuando debe registrar un turno, pasando el *logger.record* al método *Track.run()*. A continuación se evidencia que los 6 tests pasaron sin ningún problema.



¿Por qué es mejor que *Car* no sepa nada de logging o impresión?

Es mejor porque cada clase debe tener una sola responsabilidad, si Car se encarga de moverse, de imprimir en la consola y de escribir archivos como originalmente lo hacía, cualquier cambio mínimo en la forma en que se muestran los datos o en cómo se guardan los registros obligaría a modificar la clase *Car*, lo cual no es lo mejor puesto que el carro solo debería verificar su propio estado y su movimiento. Al separarlo, el código queda mucho más limpio, ordenado y fácil de editar para correcciones futuras.

¿Qué se rompería primero si un profesor te pidiera registrar los resultados como JSON en lugar de texto plano?

Lo primero que se rompería es la arquitectura del código si estuviera acoplado, para esto se tendría que ir a modificar directamente la clase *Car* para cambiar la lógica de cómo arman y escriben el formato JSON. Al tenerlo separado con una clase independiente como *RaceLogger*, se tendría que cambiar o adaptar es el método *save()* de ese logger, sin necesidad de acceder al código de los carros, la pista o el funcionamiento principal.

Ejercicio 2

En este ejercicio las clases *Vehicle*, *Car* y *Truck* no se podían modificar y las clases *Motorcycle* y *Bicycle* no se encontraban completas porque les faltaban comportamientos de movimiento adecuados, para esto había que expandir el comportamiento del sistema original a estas clases sin afectar las 3 nombradas inicialmente. Para la clase *Motorcycle* se implementó una función que le permite avanzar de manera positiva siempre pero una distancia aleatoria con cada turno, esto se hizo usando la función *random.randint(1,8)* estableciendo ese rango de velocidad.

Para la clase *Bicycle* se creó un constructor __init__ que inicializa un contador en 0 para controlar el cansancio del ciclista y el paso de tiempo o de turnos, esto para que a medida que usando *move()* el ciclista avanzará más lentamente a medida que pasaban los turnos.

```
PS C:\Users\crist\OneDrive\Documents\solid-race-lab> pytest tests/test_ex2_ocp.py -v
rootdir: C:\Users\crist\OneDrive\Documents\solid-race-lab
configfile: pytest.ini
plugins: anyio-4.13.0
collected 4 items

tests/test_ex2_ocp.py::test_car_and_truck_unchanged PASSED                [ 25%]
tests/test_ex2_ocp.py::test_motorcycle_and_bicycle_are_vehicles PASSED    [ 50%]
tests/test_ex2_ocp.py::test_motorcycle_moves_variably PASSED              [ 75%]
tests/test_ex2_ocp.py::test_bicycle_gets_slower_over_time PASSED          [100%]

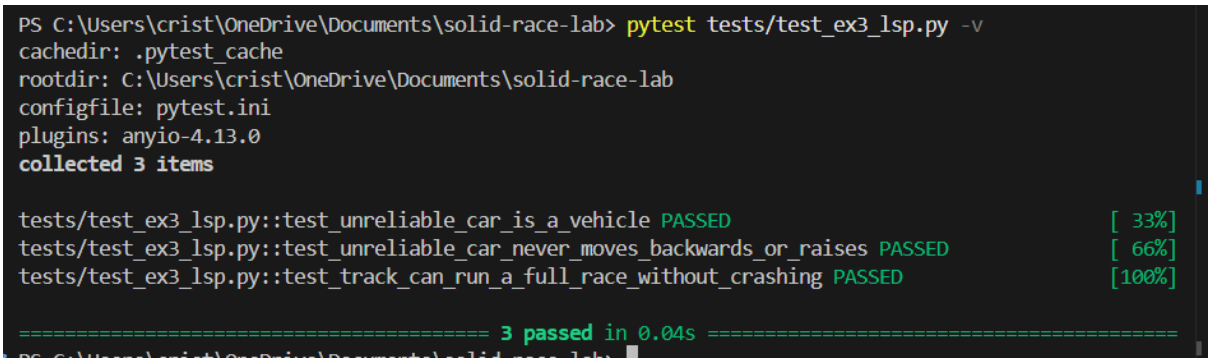
============================== 4 passed in 0.05s ==============================
```

Si tuviste que editar las clases `Vehicle` o `Track` para terminar esto, ¿qué te dice eso sobre tu diseño?

Esto dice que el diseño está rompiendo el Principio de Abierto/Cerrado (OCP), ya que está dependiendo de modificar las clases base existentes cada vez que se quiere agregar un comportamiento nuevo como las nuevas motos o bicicletas. Un diseño orientado a objetos idealmente debe estar abierto a la extensión pero cerrado a la modificación, lo que significa que deberías poder resolver nuevos requerimientos escribiendo código nuevo en subclases o módulos aparte sin tener que tocar ni arriesgar el código que ya funciona.

Ejercicio 3

Para la clase *UnreliableCar* del ejercicio 3, el problema era que a veces el carro lanzaba un error de ejecución o retrocedía en la pista, lo cual rompía el contrato básico que todo Vehicle debe cumplir: que es nunca retroceder y nunca lanzar excepciones. Para solucionar esto y cumplir con el principio de sustitución de Liskov (LSP), modificamos el método *move()* de modo que cuando ocurra la falla aleatoria, en lugar de tirar un *RuntimeError* o restar posición, simplemente no avance en ese turno y se quede quieto. De esta forma, el motor de la pista puede correr sin romperse ni necesitar parches de código defensivo, asegurando que la subclase funcione correctamente y sea totalmente sustituible.



¿Qué se habría tenido que agregar a `Track` (como `try/except` extra, delimitación de posiciones, etc.) si NO se hubiera arreglado `UnreliableCar`? ¿Por qué es mejor arreglar la subclase?

Si no se hubiera arreglado *`UnreliableCar`*, a `Track` le había tocado volverse un código lleno de parches y actualizaciones, añadiendo de manera recurrente bloques *`try/except`* para atrapar las excepciones repentinas, además de validaciones adicionales para evitar que los carros retrocedieran y dañaran la animación o la lógica de la carrera. Es mejor arreglar la subclase porque según el Principio de Sustitución de Liskov (LSP), el contrato lo debe cumplir el objeto que se sustituye, garantizando que cualquier vehículo pueda ser utilizado por el motor de la pista sin que este tenga que adivinar o filtrar errores que no le corresponden. De esta manera la pista se mantiene limpia y ordenada.

Ejercicio 4

El problema inicial era que existía una interfaz gigante llamada *`VehicleActions`* que obligaba a todos los vehículos a implementar métodos como *`refuel()`*, *`fly()`* y *`pedal_harder()`*, incluso si no tenían ningún sentido para ellos (lo que hacía que tuvieran que lanzar excepciones de métodos no soportados). Para aplicar el principio de segregación de interfaces (ISP), dividimos esa interfaz gorda en cuatro interfaces pequeñas y enfocadas: *`Movable`*, *`Refuelable`*, *`Flyable`* y *`Pedalable`*, cada una con un solo método abstracto.

Luego se ajustaron las clases *`GasCar`* y *`Bicycle`* para que heredaran únicamente de las interfaces que realmente usan (quitando los stubs innecesarios), y creamos una nueva clase llamada `Drone` que implementa solo *`Movable`* y *`Flyable`*, incluyéndola finalmente en la carrera desde el *`main()`*.

```
PS C:\Users\crist\OneDrive\Documents\solid-race-lab> pytest tests/test_ex4_isp.py -v
rootdir: C:\Users\crist\OneDrive\Documents\solid-race-lab
configfile: pytest.ini
plugins: anyio-4.13.0
collected 4 items

tests/test_ex4_isp.py::test_small_focused_interfaces_exist PASSED                    [ 25%]
tests/test_ex4_isp.py::test_gas_car_only_implements_relevant_interfaces PASSED       [ 50%]
tests/test_ex4_isp.py::test_bicycle_only_implements_relevant_interfaces PASSED       [ 75%]
tests/test_ex4_isp.py::test_drone_exists_and_only_implements_relevant_interfaces PASSED [100%]

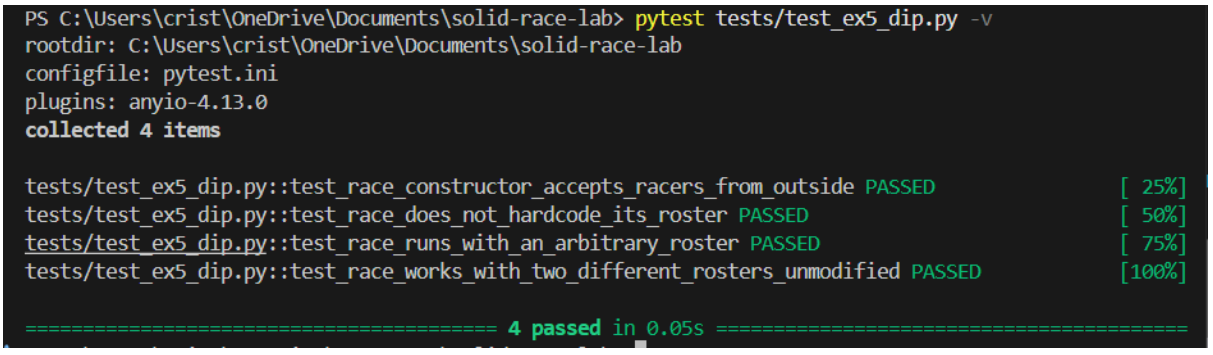
============================================ 4 passed in 0.05s =============================================
```

¿Qué interfaz necesitaría un futuro vehículo tipo Submarino o Avión? ¿Añadir uno requiere tocar alguna clase de vehículo existente?

Para un Submarino o un Avión se necesitarían interfaces específicas como *`Submersible`* o *`Flyable`*. Añadir un nuevo vehículo no requiere tocar ninguna clase existente, ya que gracias al Principio de Segregación de Interfaces (ISP), el nuevo vehículo solo implementa los métodos que realmente le sirven y hereda de las interfaces que necesita sin afectar para nada el código de los carros o bicicletas que ya estaban creados.

Ejercicio 5

Originalmente la clase *`Race`* estaba acoplada de forma rígida porque construía sus propios vehículos (*`SportsCar`* y *`DeliveryVan`*) directamente dentro de su constructor `__init__`, lo que significaba que si queríamos correr con otros vehículos tocaba modificar la clase de alto nivel por dentro. Para solucionar esto y aplicar el principio de inversión de dependencias (DIP), refactorizamos `Race` para que reciba la lista de corredores (`racers`) y opcionalmente la pista (`track`) desde el exterior por medio de inyección de dependencias, guardándolos en `self.racers` sin importar ni conocer ninguna clase de vehículo concreta. Además, creamos la nueva clase *`RocketSled`* y en el *`main()`* armamos dos rosters diferentes para correrlos con la misma clase `Race` sin tener que cambiarle nada.



¿Cómo hace que inyectar el roster sea más fácil hacerle pruebas unitarias a Race sin necesidad de una animación real en la terminal?

Al inyectar la lista de corredores (*roster*) desde el exterior, se evita que la clase *`Race`* dependa de objetos concretos o cree instancias por dentro de forma rígida. Para las

pruebas unitarias, esto lo facilita porque se pueden pasar corredores simulados (*mocks* o *stubs*) que corran en memoria de forma inmediata, sin necesidad de activar la animación real en la terminal ni depender de salidas por consola. Así, los tests se ejecutan mucho más rápido y de manera aislada optimizando su funcionamiento.

Identifica un lugar en tu propio proyecto del curso (o un proyecto pasado) donde se estuviera violando uno de estos cinco principios. ¿Qué cambiarías?

En un proyecto pasado para la clase de Ingeniería de Software I se realizó un inventario de una tienda de videojuegos donde había usuarios administrador y vendedor, y al realizarse una o varias ventas se registraba la transacción y se descontaba del inventario se identificó que se violaba el Principio de Responsabilidad Única (SRP). En el módulo de ventas había una sola clase *GestorVentas* que se encargaba de validar el stock disponible, modificar directamente las cantidades en el inventario y registrar la venta. Lo que se cambiaría es separar esas responsabilidades: crear un servicio exclusivo para el manejo del inventario como un *ServicioInventario y* otro servicio dedicado netamente a procesar la venta `ServicioVentas`. Esto porque si en el futuro cambia la forma de descontar el stock o de registrar los datos, se modifica únicamente su respectiva clase sin afectar directamente el gestor principal de las ventas.